Python Programming

Unit 1 — Study Notes

Covering: Introduction to Python · Features · Advantages · Limitations · Work Environment · Installation · Program Structure · Literals · Variables · Data Types · Input · Print Options · Operators · Conditional Statements · Looping · Errors

Topic 1: Introduction to Python

Python is a high-level, general-purpose programming language created by **Guido van Rossum** and first released in **1991**. It is designed to be easy to read and easy to write, using plain English-like syntax instead of complicated punctuation found in many other languages.

Why was Python created?

Guido van Rossum wanted a language that was simple, readable, and could be used to quickly turn ideas into working programs. The name 'Python' comes from the British comedy show *Monty Python's Flying Circus*, not the snake.

What kind of language is Python?

- **High-level language:** Close to human language, far from machine code.
- **Interpreted language:** Code is executed line-by-line by an interpreter (no separate compilation step needed before running).
- **General-purpose:** Used in web development, data science, automation, AI/ML, game development, scripting, and more.
- **Object-Oriented and Procedural:** Supports multiple programming styles (also called *multi-paradigm*).
- **Dynamically typed:** You don't need to declare the data type of a variable; Python figures it out automatically.

A simple first program

Example 1.1 — The classic first program

```
print("Hello, World!")

# Output:
# Hello, World!
```

■ Notice there are no semicolons, no curly braces, and no need to declare a 'main' function. This is what makes Python beginner-friendly.

Where is Python used today?

- Web development (Django, Flask)
- Data Science and Machine Learning (pandas, NumPy, scikit-learn, TensorFlow)
- Automation and scripting (renaming files, sending emails, web scraping)
- Game development (Pygame)
- Desktop applications (Tkinter, PyQt)
- Cybersecurity and ethical hacking scripts

Topic 2: Features of Python

Python has become one of the most popular languages in the world because of the features listed below. Each feature explains *why* Python behaves the way it does.

Simple and Easy to Learn

Python's syntax is close to plain English. There is very little boilerplate code, which makes it a great language for beginners.

Free and Open Source

Python can be downloaded and used for free. Its source code is open, meaning anyone can view, modify, and contribute to it.

Interpreted Language

Python code runs line-by-line through an interpreter. This makes debugging easier because errors are reported at the exact line where they occur.

High-Level Language

Programmers do not need to manage memory manually (like allocating/freeing memory as in C) — Python handles this automatically.

Portable / Platform Independent

A Python program written on Windows can run on Linux or macOS without any changes, as long as Python is installed.

Object-Oriented

Python supports classes, objects, inheritance, and polymorphism, allowing organized and reusable code.

Extensive Standard Library

Python comes with a large collection of built-in modules (math, os, datetime, random, etc.) for common tasks, so you don't have to write everything from scratch.

Dynamically Typed

You don't need to specify a variable's data type; Python assigns it automatically based on the value stored.

Extensible

Python can be extended using other languages like C or C++ for performance-critical tasks.

Embeddable

Python code can be embedded inside applications written in other languages to provide scripting capabilities.

Large Community and Library Support

Millions of developers use Python, and there are thousands of third-party libraries (via pip) for almost any task.

Topic 3: Advantages of Python

While 'Features' describe *what* Python is, 'Advantages' describe the practical *benefits* a programmer or company gets from using it.

Key Advantages

- **Readability:** Clean, indentation-based syntax makes code easy to read and maintain, even by someone who didn't write it.
- **Faster Development Time:** Less code is needed to accomplish the same task compared to languages like Java or C++.
- **Huge Ecosystem of Libraries:** Ready-made libraries exist for almost everything — NumPy for maths, Django for websites, Pandas for data.
- **Great for Beginners:** Simple syntax makes it an excellent first programming language to learn logic and problem-solving.
- **Strong Community Support:** Any error or doubt can usually be resolved quickly by searching online forums like Stack Overflow.
- **Cross-Platform Compatibility:** Works the same way on Windows, macOS, and Linux.
- **Integration Capabilities:** Easily integrates with other languages and technologies (C, C++, Java, databases, web APIs).
- **Used by Top Companies:** Google, Instagram, Netflix, Spotify, and NASA use Python in parts of their systems.
- **Good for Rapid Prototyping:** Ideas can be tested and turned into working demos very quickly.

■ Because Python needs fewer lines of code, a task that takes 50 lines in Java might take only 10–15 lines in Python.

Topic 4: Limitations of Python

No language is perfect. Understanding Python's limitations helps you know when it may NOT be the best choice for a particular project.

Key Limitations

- **Slower Execution Speed:** Because Python is interpreted (not compiled directly to machine code like C/C++), it generally runs slower.
- **High Memory Consumption:** Python's flexibility (dynamic typing, automatic memory management) comes at the cost of higher memory usage.
- **Not Ideal for Mobile Development:** Python is rarely used to build mobile apps; languages like Kotlin (Android) or Swift (iOS) are preferred.
- **Global Interpreter Lock (GIL):** This restricts true multi-threaded execution in a single process, which can limit performance in CPU-heavy, multi-threaded programs.
- **Runtime Errors:** Since Python is dynamically typed, some errors (like passing the wrong data type) are only caught while the program is running, not before.
- **Weak in Mobile and Browser-based Computing:** Compared to JavaScript (browser) or Java/Kotlin (Android), Python has limited native support.
- **Database Access Layer:** Python's database access layer is considered less mature/advanced compared to technologies like JDBC (Java) or ODBC.

■ Despite these limitations, Python remains one of the top choices for learning, scripting, data science, and rapid application development.

Topic 5: Python Work Environment

Before writing Python programs, it helps to understand the different tools and environments available to write, run, and manage your code.

1. Python Interpreter (Interactive Mode)

Typing **python** or **python3** in a terminal opens an interactive shell (also called the REPL — Read, Evaluate, Print, Loop) where you can type one line of code at a time and see the result immediately.

Example 5.1 — Using the interactive shell

```
>>> 2 + 3
5
>>> print("Hi")
Hi
```

2. Script Mode

In script mode, you write your full program in a file with a .py extension (e.g., program.py) and run the entire file at once.

Running a script from the terminal

```
python program.py
```

3. Integrated Development Environments (IDEs)

IDEs provide a complete environment with a code editor, debugger, and run button, making development easier. Popular choices include:

- **IDLE:** Comes bundled with Python; simple and lightweight.
- **PyCharm:** A powerful, feature-rich IDE popular for larger projects.
- **VS Code (Visual Studio Code):** A lightweight, highly customizable editor with a Python extension.
- **Jupyter Notebook:** Popular for data science; allows mixing code, text, and visual output (graphs) in one document, cell by cell.
- **Google Colab:** A free, cloud-based Jupyter-like environment that requires no installation.

4. Virtual Environments

A virtual environment is an isolated workspace that keeps a project's installed packages separate from other projects, avoiding version conflicts.

Creating and activating a virtual environment

```
python -m venv myenv

# Activate on Windows:
myenv\Scripts\activate

# Activate on macOS/Linux:
source myenv/bin/activate
```

Topic 6: Installation of Python

Step 1: Download Python

- Visit the official website: **python.org/downloads**
- Download the latest stable version suitable for your operating system (Windows / macOS / Linux).

Step 2: Install on Windows

- Run the downloaded .exe installer.
- **Very important:** Check the box that says **'Add Python to PATH'** before clicking Install — this allows you to run Python from any folder in the command prompt.
- Click 'Install Now' and wait for the setup to finish.

Step 3: Install on macOS

- macOS often comes with an older version of Python pre-installed.
- Download and run the official .pkg installer from python.org, OR
- Use Homebrew: `brew install python3`

Step 4: Install on Linux

Debian/Ubuntu-based systems

```
sudo apt update
sudo apt install python3
```

Step 5: Verify the Installation

Open a terminal or command prompt and type:

Checking the installed version

```
python --version
# or
python3 --version

# Output example:
# Python 3.12.4
```

Step 6: Verify pip (Package Installer)

pip is used to install third-party libraries. It usually comes bundled with Python.

Checking pip

```
pip --version
```

■ If 'python' is not recognized as a command, it usually means Python was not added to the system PATH during installation — simply reinstall and tick that option.

Topic 7: Python Program Structure

Unlike languages such as C or Java, Python does not require a special 'main' function to start execution — a Python file is simply executed from top to bottom.

Basic Building Blocks of a Python Program

- **Comments:** Notes for humans, ignored by the interpreter.
- **Import statements:** Bring in extra functionality from modules.
- **Variables and Data:** Store values used by the program.
- **Statements/Expressions:** Instructions the interpreter executes.
- **Functions:** Reusable blocks of code.
- **Indentation:** Defines blocks of code (used instead of { } braces).

Example: A complete small program

Example 7.1 — Structure of a simple program

```
# 1. Comment describing the program
# This program adds two numbers

# 2. Import statement (optional)
import math

# 3. Function definition
def add_numbers(a, b):
    return a + b

# 4. Main body of the program
num1 = 10
num2 = 5
result = add_numbers(num1, num2)
print("The sum is:", result)

# Output:
# The sum is: 15
```

The Importance of Indentation

Python uses **indentation** (spaces at the beginning of a line) to define which statements belong to a block, such as inside an if-statement, loop, or function. Incorrect indentation causes an **IndentationError**.

Correct vs Incorrect indentation

```
# Correct
if 5 > 2:
    print("5 is greater")

# Incorrect (will cause an error)
if 5 > 2:
print("5 is greater")
```

Comments in Python

- **Single-line comment:** starts with a hash symbol `#`
- **Multi-line comment:** written using triple quotes `'''  ...  '''` or `"""  ...  """`

Example 7.2 — Comments

```
# This is a single-line comment

"""
This is a
multi-line comment / docstring
"""
print("Comments are ignored by Python")
```

Topic 8: Literals in Python

A **literal** is a fixed value written directly in the source code — it is data as it appears, not stored in a variable yet. For example, in `x = 10`, the value **10** is a literal.

Types of Literals

1. Numeric Literals

Represent numbers, and are of three types:

Example 8.1 — Numeric literals

```
a = 10          # Integer literal
b = 3.14        # Floating-point literal
c = 2 + 5j      # Complex literal (j represents the imaginary part)
```

2. String Literals

Text enclosed in single, double, or triple quotes.

Example 8.2 — String literals

```
s1 = 'Hello'              # Single-quoted
s2 = "Hello"              # Double-quoted
s3 = '''Hello,
World'''                  # Triple-quoted (multi-line string)
```

3. Boolean Literals

Represent one of two truth values: `True` or `False` (note the capital first letter).

Example 8.3 — Boolean literals

```
is_active = True
is_deleted = False
```

4. Special Literal: None

`None` represents the absence of a value (similar to 'null' in other languages).

Example 8.4 — None literal

```
data = None
print(data)   # Output: None
```

5. Collection Literals

Used to create built-in data structures directly:

Example 8.5 — Collection literals

```
my_list  = [1, 2, 3]          # List literal
my_tuple = (1, 2, 3)          # Tuple literal
my_dict  = {'a': 1, 'b': 2}   # Dictionary literal
my_set   = {1, 2, 3}          # Set literal
```

6. Numeric Literals in Different Bases

Example 8.6 — Binary, Octal, Hexadecimal

```
binary_num = 0b1010     # Binary (base 2) -> equals 10
octal_num  = 0o17       # Octal (base 8)  -> equals 15
hex_num    = 0x1F       # Hexadecimal (base 16) -> equals 31
```

Topic 9: Variables in Python

A **variable** is a name given to a memory location used to store data. Think of it as a labeled box that holds a value which can change during the program's execution.

Declaring and Assigning Variables

Python does not require you to declare a variable's type in advance. A variable is created the moment you first assign a value to it, using the `=` (assignment) operator.

Example 9.1 — Creating variables

```
name = "Alice"
age = 21
height = 5.6
print(name, age, height)

# Output:
# Alice 21 5.6
```

Rules for Naming Variables

- Must start with a letter (a–z, A–Z) or an underscore (_) — never a digit.
- Can contain letters, digits, and underscores after the first character.
- Cannot contain spaces or special symbols (@, #, %, etc.).
- Cannot be a Python reserved keyword (e.g., `if`, `for`, `class`).
- Case-sensitive: `age`, `Age`, and `AGE` are three different variables.

Example 9.2 — Valid vs invalid variable names

```
# Valid names
_name = "John"
age1 = 25
total_marks = 90

# Invalid names (will cause errors)
1age = 25        # cannot start with a digit
total-marks = 90 # hyphen not allowed
class = "A"      # 'class' is a reserved keyword
```

Multiple Assignment

Python allows assigning values to multiple variables in one line:

Example 9.3 — Multiple assignment

```
a, b, c = 10, 20, 30
print(a, b, c)     # Output: 10 20 30

x = y = z = 100    # all three variables get the same value
print(x, y, z)     # Output: 100 100 100
```

Dynamic Typing

A Python variable can be reassigned to a value of a completely different data type at any point.

Example 9.4 — Dynamic typing

```
value = 10        # value is currently an integer
print(type(value))    # <class 'int'>

value = "Hello"    # now value is a string
print(type(value))    # <class 'str'>
```

Topic 10: Data Types in Python

A **data type** tells Python what kind of value a variable holds, and what operations can be performed on it. Python has several built-in data types, grouped as follows:

Category	Data Type	Example
Numeric	int	10, -5, 2024
Numeric	float	3.14, -0.5
Numeric	complex	2 + 3j
Text	str	"Hello", 'Python'
Boolean	bool	True, False
Sequence	list	[1, 2, 3]
Sequence	tuple	(1, 2, 3)
Sequence	range	range(5)
Mapping	dict	{'a': 1, 'b': 2}
Set	set	{1, 2, 3}
Set	frozenset	frozenset({1,2})
None Type	NoneType	None

1. Numeric Types

Example 10.1 — int, float, complex

```
a = 25          # int
b = 25.75       # float
c = 3 + 4j      # complex
print(type(a), type(b), type(c))
```

2. String (str)

A sequence of characters enclosed in quotes.

Example 10.2 — Strings

```
greeting = "Hello, Python!"
print(greeting[0])      # H  (first character)
print(len(greeting))    # 14 (length of string)
```

3. Boolean (bool)

Example 10.3 — Boolean

```
is_valid = True
print(type(is_valid))   # <class 'bool'>
```

4. List

An ordered, changeable (mutable) collection that allows duplicates.

Example 10.4 — List

```
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")
print(fruits)   # ['apple', 'banana', 'cherry', 'mango']
```

5. Tuple

An ordered collection that cannot be changed once created (immutable).

Example 10.5 — Tuple

```
coordinates = (10, 20)
print(coordinates[0])   # 10
```

6. Dictionary (dict)

Stores data as key–value pairs.

Example 10.6 — Dictionary

```
student = {"name": "Riya", "age": 20}
print(student["name"])   # Riya
```

7. Set

An unordered collection that automatically removes duplicate values.

Example 10.7 — Set

```
numbers = {1, 2, 2, 3, 3, 3}
print(numbers)   # {1, 2, 3}
```

Checking a Variable's Data Type

Using the type() function

```
x = 10
print(type(x))    # <class 'int'>
```

Type Conversion (Type Casting)

You can convert a value from one data type to another using functions like `int()`, `float()`, and `str()`.

Example 10.8 — Type conversion

```
a = "25"
b = int(a)        # converts string to integer
print(b + 5)      # 30

c = str(100)      # converts integer to string
print(c + " apples")   # 100 apples
```

Topic 11: Inputting Values in Python

Programs often need to take data from the user while running. Python provides the built-in `input()` function for this.

Basic Usage of input()

The `input()` function always returns the entered value as a **string**, regardless of what the user types.

Example 11.1 — Taking input

```
name = input("Enter your name: ")
print("Hello,", name)

# Sample run:
# Enter your name: Aman
# Hello, Aman
```

Taking Numeric Input

Since input() always returns a string, you must convert it using `int()` or `float()` if you plan to do arithmetic with it.

Example 11.2 — Numeric input

```
age = int(input("Enter your age: "))
print("Next year you will be:", age + 1)

height = float(input("Enter your height in meters: "))
print("Your height is:", height)
```

■ If you forget to convert the input and try to do maths directly on it, Python will raise a TypeError, since you cannot add a number to text.

Taking Multiple Inputs in One Line

Example 11.3 — Multiple inputs using split()

```
x, y = input("Enter two numbers separated by space: ").split()
x = int(x)
y = int(y)
print("Sum:", x + y)

# Sample run:
# Enter two numbers separated by space: 5 10
# Sum: 15
```

Taking a List of Numbers as Input

Example 11.4 — Using map() for multiple values

```
numbers = list(map(int, input("Enter numbers: ").split()))
print(numbers)

# Sample run:
# Enter numbers: 1 2 3 4
# [1, 2, 3, 4]
```

Topic 12: Printing in Python (with its Various Options)

The `print()` function is used to display output on the screen. It is one of the most frequently used functions in Python and has several useful options.

1. Basic Printing

Example 12.1 — Basic print

```
print("Hello, World!")
print(10)
print(3.14)
```

2. Printing Multiple Values

Multiple values can be printed together by separating them with commas; Python automatically adds a space between them.

Example 12.2 — Multiple values

```
name = "Sam"
age = 22
print("Name:", name, "Age:", age)

# Output:
# Name: Sam Age: 22
```

3. The 'sep' Parameter (Separator)

By default, print() separates values with a single space. This can be changed using the `sep` parameter.

Example 12.3 — Using sep

```
print("2024", "01", "15", sep="-")
# Output: 2024-01-15

print("apple", "banana", "cherry", sep=", ")
# Output: apple, banana, cherry
```

4. The 'end' Parameter

By default, print() moves to a new line after printing. The `end` parameter changes what is added at the end instead of a newline.

Example 12.4 — Using end

```
print("Hello", end=" ")
print("World")
# Output: Hello World   (printed on the same line)

print("Loading", end="...")
print("Done")
# Output: Loading...Done
```

5. Formatting Output with f-strings (Recommended)

f-strings (formatted string literals) let you embed variables directly inside a string using curly braces, prefixed with the letter `f`.

Example 12.5 — f-strings

```
name = "Neha"
marks = 89.5
print(f"{name} scored {marks} marks.")
# Output: Neha scored 89.5 marks.

print(f"Marks rounded: {marks:.0f}")   # controlling decimal places
# Output: Marks rounded: 90
```

6. Formatting Output with format() Method

Example 12.6 — .format() method

```
print("{} scored {} marks.".format("Neha", 89.5))
# Output: Neha scored 89.5 marks.

print("{0} is {1} years old".format("Raj", 20))
# Output: Raj is 20 years old
```

7. The Old % Formatting (Legacy Style)

Example 12.7 — % formatting

```
name = "Tom"
age = 30
print("%s is %d years old" % (name, age))
# Output: Tom is 30 years old
```

8. Printing with Escape Sequences

Escape Sequence	Meaning
\\n	New line
\\t	Tab space
\\\\	Backslash character
\\'	Single quote
\\"	Double quote

Example 12.8 — Escape sequences

```
print("Line1\nLine2")
# Output:
# Line1
# Line2

print("Name:\tAge")
# Output: Name:    Age
```

Topic 13: Operators in Python

Operators are special symbols used to perform operations on variables and values. Python groups operators into several categories.

1. Arithmetic Operators

Operator	Meaning	Example (a=10, b=3)
+	Addition	a + b -> 13
-	Subtraction	a - b -> 7
*	Multiplication	a * b -> 30
/	Division (float result)	a / b -> 3.333...
//	Floor Division (integer result)	a // b -> 3
%	Modulus (remainder)	a % b -> 1
**	Exponent (power)	a ** b -> 1000

2. Comparison (Relational) Operators

Used to compare two values; the result is always True or False.

Operator	Meaning	Example (a=10, b=3)
==	Equal to	a == b -> False
!=	Not equal to	a != b -> True
>	Greater than	a > b -> True
<	Less than	a < b -> False
>=	Greater than or equal to	a >= b -> True
<=	Less than or equal to	a <= b -> False

3. Logical Operators

Used to combine multiple conditions.

Operator	Meaning	Example
and	True only if both conditions are True	(5>2) and (3>1) -> True
or	True if at least one condition is True	(5>2) or (3<1) -> True
not	Reverses the result	not(5>2) -> False

4. Assignment Operators

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
//=	x //= 3	x = x // 3
%=	x %= 3	x = x % 3
**=	x **= 3	x = x ** 3

Topic 13: Operators in Python (continued)

5. Identity Operators

Used to check whether two variables point to the same object in memory (not just equal values).

Example 13.1 — is, is not

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a is b)       # True  (b refers to the same object as a)
print(a is c)       # False (c is a different object, though values match)
print(a == c)       # True  (values are equal)
```

6. Membership Operators

Used to check whether a value exists within a sequence (list, tuple, string, etc.).

Example 13.2 — in, not in

```
fruits = ["apple", "banana", "cherry"]
print("banana" in fruits)       # True
print("mango" not in fruits)    # True
```

7. Bitwise Operators

Operate directly on the binary representation of numbers.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift

Example 13.3 — Bitwise operators

```
a = 5      # binary: 0101
b = 3      # binary: 0011
print(a & b)   # 1  (0001)
print(a | b)   # 7  (0111)
print(a ^ b)   # 6  (0110)
```

Operator Precedence

When multiple operators appear in one expression, Python follows a fixed order (similar to BODMAS in mathematics): Parentheses first, then exponent (**), then multiplication/division/modulus, then addition/subtraction, and finally comparison and logical operators.

Example 13.4 — Precedence

```
result = 10 + 2 * 3
print(result)     # 16, not 36 (multiplication happens first)

result2 = (10 + 2) * 3
print(result2)    # 36 (parentheses change the order)
```

Topic 14: Conditional Statements in Python

Conditional statements allow a program to make decisions and execute different blocks of code depending on whether a condition is True or False.

1. The if Statement

Example 14.1 — Simple if

```
age = 20
if age >= 18:
    print("You are eligible to vote.")

# Output: You are eligible to vote.
```

2. The if-else Statement

Example 14.2 — if-else

```
age = 15
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")

# Output: Not eligible to vote
```

3. The if-elif-else Ladder

Used when there are more than two possible outcomes.

Example 14.3 — if-elif-else

```
marks = 72

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Fail")

# Output: Grade C
```

4. Nested if Statements

An if statement placed inside another if statement.

Example 14.4 — Nested if

```
num = 15

if num > 0:
    if num % 2 == 0:
        print("Positive and Even")
    else:
        print("Positive and Odd")
else:
```

```
    print("Non-positive number")

# Output: Positive and Odd
```

5. Short-hand if (Ternary / Conditional Expression)

A compact way to write a simple if-else in a single line.

Example 14.5 — Ternary expression

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)     # Output: Adult
```

6. Using Logical Operators in Conditions

Example 14.6 — Combining conditions

```
age = 25
has_id = True

if age >= 18 and has_id:
    print("Entry allowed")
else:
    print("Entry denied")

# Output: Entry allowed
```

■ Remember: In Python, indentation is not optional inside conditional blocks — it is how Python knows which lines belong to the if/elif/else.

Topic 15: Looping in Python

Loops are used to execute a block of code repeatedly, either a fixed number of times or until a certain condition is no longer true. This avoids writing the same code again and again.

1. The for Loop

Used when the number of repetitions is known in advance, or to iterate over a sequence (list, string, range, etc.).

Example 15.1 — for loop with range()

```
for i in range(1, 6):
    print(i)

# Output:
# 1
# 2
# 3
# 4
# 5
```

Example 15.2 — for loop over a list

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Output:
# apple
# banana
# cherry
```

Understanding range()

Syntax	Meaning	Example
range(stop)	0 up to (stop - 1)	range(5) -> 0,1,2,3,4
range(start, stop)	start up to (stop - 1)	range(2,6) -> 2,3,4,5
range(start, stop, step)	start to stop, jumping by step	range(0,10,2) -> 0,2,4,6,8

2. The while Loop

Used when the number of repetitions is not known in advance — the loop continues as long as a condition remains True.

Example 15.3 — while loop

```
count = 1
while count <= 5:
    print(count)
    count += 1

# Output:
# 1
# 2
```

```
# 3
# 4
# 5
```

■ Always make sure something inside a while loop changes (like count += 1) — otherwise it becomes an infinite loop that never stops.

3. The break Statement

Immediately exits the loop, even if the loop condition is still True.

Example 15.4 — break

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)

# Output:
# 1
# 2
# 3
# 4
```

4. The continue Statement

Skips the current iteration and moves to the next one.

Example 15.5 — continue

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)

# Output:
# 1
# 2
# 4
# 5   (3 is skipped)
```

5. The pass Statement

A placeholder that does nothing — used when a statement is syntactically required but no action is needed yet.

Example 15.6 — pass

```
for i in range(5):
    if i == 3:
        pass    # do nothing for now
    print(i)
```

6. Nested Loops

A loop placed inside another loop.

Example 15.7 — Nested for loop

```
for i in range(1, 3):
    for j in range(1, 3):
        print(f"i={i}, j={j}")
```

```
# Output:
# i=1, j=1
# i=1, j=2
# i=2, j=1
# i=2, j=2
```

7. The else Clause with Loops

Python allows an else block after a loop, which runs only if the loop completes WITHOUT hitting a break statement.

Example 15.8 — for-else

```
for i in range(1, 4):
    print(i)
else:
    print("Loop finished normally")

# Output:
# 1
# 2
# 3
# Loop finished normally
```

Topic 16: Errors in Python

An **error** is a problem in a program that prevents it from running correctly, or that stops it partway through. Understanding the common types of errors — and how to read the message Python gives you — helps in debugging (finding and fixing problems) much faster.

Four Common Types of Errors

1. IndentationError

Occurs when the spacing at the start of a line does not match what Python expects. Since Python uses indentation to define blocks of code (inside if statements, loops, functions, etc.), inconsistent or missing indentation breaks the structure of the program.

Example 16.1 — IndentationError

```
if 5 > 2:
print("5 is greater")   # not indented under the if

# Error:
# IndentationError: expected an indented block after 'if' statement
```

Correct version:

Fixed version

```
if 5 > 2:
    print("5 is greater")   # properly indented
```

2. NameError

Occurs when the program tries to use a variable or function name that has not been defined yet — often caused by a typo or by using a variable before assigning it a value.

Example 16.2 — NameError

```
print(total_marks)

# Error:
# NameError: name 'total_marks' is not defined
```

3. TypeError

Occurs when an operation is performed on a data type that does not support it — for example, trying to add a number and a string directly.

Example 16.3 — TypeError

```
age = 20
print("Your age is " + age)

# Error:
# TypeError: can only concatenate str (not "int") to str
```

Correct version:

Fixed version

```
age = 20
print("Your age is " + str(age))   # convert int to str first
```

4. ValueError

Occurs when a function receives an argument of the correct data type, but with an inappropriate or invalid value — for example, trying to convert a non-numeric string into an integer.

Example 16.4 — ValueError

```
num = int("twenty")

# Error:
# ValueError: invalid literal for int() with base 10: 'twenty'
```

Error	Typical Cause
IndentationError	Incorrect or missing indentation of a code block
NameError	Using a variable/function that was never defined (often a typo)
TypeError	Using an operation on incompatible/wrong data types
ValueError	Right data type, but an invalid value for that operation

Reading a Traceback

When an error occurs, Python does not just print the error name — it prints a **traceback**: a report showing exactly where and how the error happened. Learning to read a traceback quickly is one of the most useful debugging skills.

Example 16.5 — A sample traceback

```
def calculate_average(marks):
    total = sum(marks)
    return total / len(marks)

def show_result():
    marks = []
    print(calculate_average(marks))

show_result()
```

The traceback this program produces

```
Traceback (most recent call last):
  File "report.py", line 8, in <module>
    show_result()
  File "report.py", line 6, in show_result
    print(calculate_average(marks))
  File "report.py", line 3, in calculate_average
    return total / len(marks)
ZeroDivisionError: division by zero
```

How to Read It — Step by Step

- **Start reading from the bottom, not the top.** The very last line always tells you the **type of error** and a short **message** describing what went wrong — this is the fastest way to know what happened.
- **The line just above it** shows the exact piece of code that was running when the error occurred (here: `return total / len(marks)`).
- **'File ... line ..., in ...'** tells you the file name, the line number, and which function the error occurred inside — this pinpoints exactly where to look in your code.
- **The traceback is read bottom-to-top as a call chain:** the topmost block shows where the program started (`show_result()` was called first), and each block below it shows the next function that was entered, until we reach the exact line that finally failed.

- **'Traceback (most recent call last)'** is simply a heading that confirms the order — the deepest, most recent call is shown last, right above the final error line.

■ A simple approach for beginners: read the last line first to know 'what' went wrong, then look at the 'File ..., line ...' just above it to know exactly 'where' it went wrong in your own code.

Quick Practice: Match the Traceback to the Error

Example 16.6 — Another traceback to analyse

```
Traceback (most recent call last):
  File "marks.py", line 4, in <module>
    percentage = marks / total
NameError: name 'total' is not defined
```

Here, the last line immediately tells us it is a **NameError**, and the line above shows that the variable `total` was used on line 4 without ever being defined earlier in the program.

Prepared by Prof. Ritisha Parmar